

```

#ifndef MONTAIGNE_ASSETLOADER_HPP
#define MONTAIGNE_ASSETLOADER_HPP

#include "Mesh.hpp"
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#include <string>
#include <vector>
#include <fstream>
#include <sstream>
#include <iostream>

struct ObjVertex { float x, y, z; };
struct ObjTexCoord { float u, v; };
struct ObjNormal { float nx, ny, nz; };

class AssetLoader {
private:
    static void parseObjToken(const std::string& token, int& v, int& vt, int& vn) {
        v = vt = vn = 0;
        size_t firstSlash = token.find('/');
        if (firstSlash == std::string::npos) {
            v = std::stoi(token);
            return;
        }
        v = std::stoi(token.substr(0, firstSlash));
        size_t secondSlash = token.find('/', firstSlash + 1);
        if (secondSlash == std::string::npos) {
            std::string vtStr = token.substr(firstSlash + 1);
            if (!vtStr.empty()) vt = std::stoi(vtStr);
            return;
        }
        std::string vtStr = token.substr(firstSlash + 1, secondSlash - (firstSlash + 1));
        if (!vtStr.empty()) vt = std::stoi(vtStr);
        std::string vnStr = token.substr(secondSlash + 1);
        if (!vnStr.empty()) vn = std::stoi(vnStr);
    }

public:
    static GLuint loadTexture(const char* path) {
        GLuint texID;
        glGenTextures(1, &texID);
        glBindTexture(GL_TEXTURE_2D, texID);

        // Use standard linear filtering to bypass strict Linux driver NPOT/mipmap constraints
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

        // Path fallback utility
        std::string chosenPath = path;
        std::ifstream testFile(chosenPath);
        if (!testFile.is_open()) {
            chosenPath = "models/" + std::string(path);
        } else {
            testFile.close();
        }

        int w, h, channels;
        unsigned char* data = stbi_load(chosenPath.c_str(), &w, &h, &channels, 0);
        if (data) {
            GLenum format = (channels == 4) ? GL_RGBA : GL_RGB;

```

```

    glTexImage2D(GL_TEXTURE_2D, 0, format, w, h, 0, format, GL_UNSIGNED_BYTE, data);
    std::cout << "[AssetLoader] Successfully loaded texture: " << chosenPath << " (" << w <<
    "x" << h << ")\n";
    stbi_image_free(data);
} else {
    std::cout << "[AssetLoader] Warning: Could not load image file " << path << ". Using
    white fallback.\n";
    unsigned char whitePixel[] = { 255, 255, 255 };
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, 1, 1, 0, GL_RGB, GL_UNSIGNED_BYTE, whitePixel);
}
return texID;
}

static Mesh loadMesh(const std::string& path) {
    std::vector<ObjVertex> temp_vertices;
    std::vector<ObjTexCoord> temp_uvs;
    std::vector<ObjNormal> temp_normals;
    std::vector<float> final_buffer;

    auto setupVAO = [](const float* vertices, size_t size, int count) -> Mesh {
        GLuint vao, vbo;
        glGenVertexArrays(1, &vao);
        glGenBuffers(1, &vbo);

        glBindVertexArray(vao);
        glBindBuffer(GL_ARRAY_BUFFER, vbo);
        glBufferData(GL_ARRAY_BUFFER, size, vertices, GL_STATIC_DRAW);

        // Position attribute
        glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)0);
        glEnableVertexAttribArray(0);
        // UV attribute
        glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(3 *
        sizeof(float)));
        glEnableVertexAttribArray(1);
        // Normal attribute
        glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 8 * sizeof(float), (void*)(5 *
        sizeof(float)));
        glEnableVertexAttribArray(2);

        glBindVertexArray(0);
        return { vao, vbo, count };
    };

    std::ifstream file(path);
    if (!file.is_open()) {
        std::cout << "[AssetLoader] Warning: " << path << " missing. Serving fallback cube
        geometry.\n";
        float fallback[] = {
            -0.5f,-0.5f, 0.5f, 0.0f,0.0f, 0.0f,0.0f, 1.0f, 0.5f,-0.5f, 0.5f, 1.0f,0.0f, 0.0f,
            0.0f, 1.0f, 0.5f, 0.5f, 0.5f, 1.0f,1.0f, 0.0f,0.0f, 1.0f,
            0.5f, 0.5f, 0.5f, 1.0f,1.0f, 0.0f,0.0f, 1.0f, -0.5f, 0.5f, 0.5f, 0.0f,1.0f, 0.0f,
            0.0f, 1.0f, -0.5f,-0.5f, 0.5f, 0.0f,0.0f, 0.0f,0.0f, 1.0f,
            -0.5f,-0.5f,-0.5f, 0.0f,0.0f, 0.0f,0.0f,-1.0f, -0.5f, 0.5f,-0.5f, 0.0f,1.0f, 0.0f,
            0.0f,-1.0f, 0.5f, 0.5f,-0.5f, 1.0f,1.0f, 0.0f,0.0f,-1.0f,
            0.5f, 0.5f,-0.5f, 1.0f,1.0f, 0.0f,0.0f,-1.0f, 0.5f,-0.5f,-0.5f, 1.0f,0.0f, 0.0f,
            0.0f,-1.0f, -0.5f,-0.5f,-0.5f, 0.0f,0.0f, 0.0f,0.0f,-1.0f,
            -0.5f, 0.5f, 0.5f, 1.0f,0.0f,-1.0f, 0.0f, 0.0f, -0.5f, 0.5f,-0.5f, 1.0f,1.0f,-1.0f,
            0.0f, 0.0f, -0.5f,-0.5f,-0.5f, 0.0f,1.0f,-1.0f, 0.0f, 0.0f,
            -0.5f,-0.5f,-0.5f, 0.0f,1.0f,-1.0f, 0.0f, 0.0f, -0.5f,-0.5f, 0.5f, 0.0f,0.0f,-1.0f,
            0.0f, 0.0f, -0.5f, 0.5f, 0.5f, 1.0f,0.0f,-1.0f, 0.0f, 0.0f,
            0.5f, 0.5f, 0.5f, 1.0f,0.0f, 1.0f, 0.0f, 0.0f, 0.5f,-0.5f, 0.5f, 0.0f,0.0f, 1.0f,
            0.0f, 0.0f, 0.5f,-0.5f,-0.5f, 0.0f,1.0f, 1.0f, 0.0f, 0.0f,

```

```

    0.5f,-0.5f,-0.5f, 0.0f,1.0f, 1.0f,0.0f, 0.0f, 0.5f, 0.5f,-0.5f, 1.0f,1.0f, 1.0f,
    0.0f, 0.0f, 0.5f, 0.5f, 0.5f, 1.0f,0.0f, 1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f,-0.5f, 0.0f,1.0f, 0.0f, 1.0f, 0.0f, -0.5f, 0.5f, 0.5f, 0.0f,0.0f, 0.0f,
    1.0f, 0.0f, 0.5f, 0.5f, 0.5f, 1.0f,0.0f, 1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f,0.0f, 0.0f, 1.0f, 0.0f, 0.5f, 0.5f,-0.5f, 1.0f,1.0f, 0.0f,
    1.0f, 0.0f, -0.5f, 0.5f,-0.5f, 0.0f,1.0f, 0.0f, 1.0f, 0.0f,
    -0.5f,-0.5f,-0.5f, 0.0f,1.0f, 0.0f,-1.0f, 0.0f, 0.5f,-0.5f,-0.5f, 1.0f,1.0f,
    0.0f,-1.0f, 0.0f, 0.5f,-0.5f, 0.5f, 1.0f,0.0f, 0.0f,-1.0f, 0.0f,
    0.5f,-0.5f, 0.5f, 1.0f,0.0f, 0.0f,-1.0f, 0.0f, -0.5f,-0.5f, 0.5f, 0.0f,0.0f,
    0.0f,-1.0f, 0.0f, -0.5f,-0.5f,-0.5f, 0.0f,1.0f, 0.0f,-1.0f, 0.0f
};
return setupVAO(fallback, sizeof(fallback), 36);
}

```

```

std::string line;
while (std::getline(file, line)) {
    if (line.substr(0, 2) == "v ") {
        std::istringstream s(line.substr(2));
        ObjVertex v; s >> v.x >> v.y >> v.z;
        temp_vertices.push_back(v);
    } else if (line.substr(0, 3) == "vt ") {
        std::istringstream s(line.substr(3));
        ObjTexCoord vt; s >> vt.u >> vt.v;
        temp_uvs.push_back(vt);
    } else if (line.substr(0, 3) == "vn ") {
        std::istringstream s(line.substr(3));
        ObjNormal vn; s >> vn.nx >> vn.ny >> vn.nz;
        temp_normals.push_back(vn);
    } else if (line.substr(0, 2) == "f ") {
        std::istringstream s(line.substr(2));
        std::string token;
        std::vector<std::string> tokens;
        while (s >> token) {
            tokens.push_back(token);
        }

        // Quad-to-triangle triangulation handler
        for (size_t i = 1; i < tokens.size() - 1; ++i) {
            std::string faceTokens[3] = { tokens[0], tokens[i], tokens[i + 1] };
            for (int j = 0; j < 3; ++j) {
                int vIdx = 0, uvIdx = 0, nIdx = 0;
                parseObjToken(faceTokens[j], vIdx, uvIdx, nIdx);

                if (vIdx > 0 && vIdx <= (int)temp_vertices.size()) {
                    ObjVertex vert = temp_vertices[vIdx - 1];
                    final_buffer.push_back(vert.x); final_buffer.push_back(vert.y);
                    final_buffer.push_back(vert.z);
                } else {
                    final_buffer.push_back(0.0f); final_buffer.push_back(0.0f);
                    final_buffer.push_back(0.0f);
                }

                if (uvIdx > 0 && uvIdx <= (int)temp_uvs.size()) {
                    ObjTexCoord uv = temp_uvs[uvIdx - 1];
                    final_buffer.push_back(uv.u); final_buffer.push_back(uv.v);
                } else {
                    final_buffer.push_back(j == 0 ? 0.0f : (j == 1 ? 1.0f : 1.0f));
                    final_buffer.push_back(j == 0 ? 0.0f : (j == 1 ? 0.0f : 1.0f));
                }

                if (nIdx > 0 && nIdx <= (int)temp_normals.size()) {
                    ObjNormal norm = temp_normals[nIdx - 1];
                    final_buffer.push_back(norm.nx); final_buffer.push_back(norm.ny);

```

```
        final_buffer.push_back(norm.nz);
    } else {
        final_buffer.push_back(0.0f); final_buffer.push_back(1.0f);
        final_buffer.push_back(0.0f);
    }
}
}
}

int totalVertices = static_cast<int>(final_buffer.size() / 8);
std::cout << "[AssetLoader] Successfully parsed " << path << " (" << totalVertices << "
indices processed)\n";

return setupVAO(final_buffer.data(), final_buffer.size() * sizeof(float), totalVertices);
}
};

#endif
```